Full Stack Development With MERN
Project Documentation: Nutrition Assistant

1. Introduction

Project Title: Nutrition Assistant

Team Members:

Student Name : Akula Munilokthisri

Role : Full Stack Developer

Responsibilities : Setting up client folder for frontend, Setting up Server folder for backend, Backend Structure, Development and Explanation of Backend

Student Name : T Manohar

Role : Full Stack Developer

Responsibilities : Technical Architecture, ER Diagram, Key Features, Roles and Responsibility, User Flow, MVC pattern, Creating Project Folder

Student Name : T ArunKumar

Role : Full Stack Developer

Responsibilities : Configure MongoDB, Create Database Connection, Create Schemas and Models

Student Name : Devireddi Krishna Rupa

Role : Full Stack Developer

Responsibilities : Frontend Structure, Development and Explanation of Frontend

Student Name : Bhumana Pravallika

Role : Full Stack Developer

Responsibilities : Steps for Execution, Demo Screenshots

2. Project Overview

Purpose:

Nutrition Assistant is a MERN stack web application designed to help users generate personalized nutrition suggestions. The application allows users to register, log in, enter personal health details, and receive a suggested nutrition plan with daily calories, macros, and meal recommendations.

Features:

- User registration and login
- JWT-based authentication
- Protected user profile
- Personalized nutrition plan generation
- Daily calorie calculation
- Protein, carbs, and fats calculation
- Meal-wise nutrition suggestions
- MongoDB database storage
- Responsive React frontend
- Express backend serving both API and frontend

3. Architecture

Frontend:

The frontend is built using React with Vite. React Router is used for page navigation. Axios is used to communicate with backend APIs. The frontend contains pages for landing, login, registration, user profile, creating nutrition plans, and viewing suggested nutrition.

Backend:

The backend is developed using Node.js and Express.js. It handles API routes, user authentication, JWT token generation, protected routes, user profile management, and nutrition suggestion generation.

Database:

MongoDB is used as the database with Mongoose for schema creation and database interaction. The main collections are Users and Suggestions.

4. Setup Instructions

Prerequisites:

- Node.js
- npm
- MongoDB or MongoDB Atlas
- VS Code
- Web browser

Installation:

Open the project folder in VS Code.

Install all dependencies:

npm run install:all

Create .env inside the server folder:

PORT=9000

MONGO_URI=your_mongodb_connection_string

JWT_SECRET=your_secret_key

Build the frontend:

npm run build

Start the application:

npm start

5. Folder Structure

Client:

client/ contains the React frontend.

src/ contains all frontend source code.
components/ contains reusable UI components like Navbar, Footer, and MealCard.
pages/ contains application pages such as LandingPage, Login, Register, Home, UserData, NewPlan, and SuggestedNutrition.

assets/ contains images and static assets.
vite.config.js contains Vite configuration.

Server:

server/ contains the Express backend.

server.js is the main backend entry file.
models/ contains MongoDB schemas such as User and Suggestion.
controllers/ contains backend logic for users and suggestions.
routes/ contains API route definitions.
middlewares/ contains authentication middleware.
utils/ contains nutrition calculation logic.
db/config.js handles MongoDB connection.

6. Running The Application

From the main project folder:

npm run build

npm start

The application runs at:

http://localhost:9000

API check URL:

http://localhost:9000/api

7. API Documentation

User Routes

POST /api/users/register
Registers a new user.

Request body:

```
{
  "name": "John",
  "email": "john@example.com",
  "password": "123456"
```

}

POST /api/users/login
Logs in an existing user.

Request body:

{

"email": "john@example.com",

"password": "123456"

}

GET /api/users/profile
Fetches logged-in user profile.
Requires Bearer token.

Suggestion Routes

POST /api/suggestions/generate
Generates a nutrition suggestion.
Requires Bearer token.

Request body:

{

"age": 22,

"gender": "male",

"weight": 70,

"height": 175,

"goal": "gain",

"activityLevel": "moderate"

}

GET /api/suggestions/latest
Fetches the latest generated nutrition plan.

GET /api/suggestions/history
Fetches all previous nutrition suggestions of the user.

8. Authentication

Authentication is handled using JSON Web Tokens. When a user logs in, the backend generates a JWT token. This token is stored in browser local storage and sent in the request header for protected routes.

Example header:

Authorization: Bearer token_here

Protected routes use middleware to verify the token before allowing access.

9. User Interface

The user interface includes:

- Landing page
- Login page
- Register page
- Home/dashboard page
- Profile page
- Nutrition plan form
- Suggested nutrition result page
- Meal cards showing food items, calories, and macros

10. Testing

Testing was done manually by:

- Registering a new user
- Logging in with valid credentials
- Checking protected profile access
- Generating a nutrition suggestion
- Verifying calories and macro output
- Checking MongoDB data storage
- Testing frontend-backend API communication

11. Screenshots Or Demo

Screenshots can be added for:

- Landing page
- Login page
- Register page
- Generate Nutrition Plan page
- Suggested Nutrition page
- User profile page

12. Known Issues

- Application requires a valid MongoDB connection string.
- The backend must be restarted after changes in server files.
- If port 9000 is already used, the old Node process must be stopped.

13. Future Enhancements

- Add diet preference options such as vegetarian, vegan, or non-vegetarian.
- Add BMI calculation.
- Add weekly meal plans.
- Add calorie tracking history.
- Add admin dashboard.
- Add password reset functionality.
- Add charts for nutrition progress.
- Improve UI with more interactive components.